Université de Caen Normandie
M1 Informatique – Algorithmique et Systèmes Intelligents

Projet MVC

Design Patterns et Application Interactive

Jeu d'occupation maximale de formes

Code source :
github.com/britek2001/MVC

Réalisé par :

Mohammed Yassine Britel
Abdelkerim Hamit Mahamat
Taiwan Li

Encadrant :

Yann Mathet

Année universitaire 2025 – 2026

Table des matières

Introduction **3**

1 Analyse du besoin **4**
1.1 Fonctionnalités principales . . . 4
1.2 Fonctionnalités détaillées . . . 4
1.2.1 Gestion des formes . . . 4
1.2.2 Contraintes et validation . . . 4
1.2.3 Niveaux de jeu . . . 5
1.2.4 Stratégies de génération des formes rouges . . . 5
1.2.5 Annulation et rétablissement (*Undo/Redo*) . . . 5
1.2.6 Calcul du score . . . 5
1.2.7 Timer et disparition des formes rouges . . . 5

2 Étude fonctionnelle **6**

3 Conception générale **7**

4 Conception détaillée **8**
4.1 Pattern Observer — Pour synchronisation Modèle-Vue . . . 8
4.2 Pattern State — Gestion des interactions souris . . . 8
4.3 Pattern Command — Undo/Redo . . . 9
4.4 Pattern Strategy — Génération des formes rouges . . . 9
4.5 Pattern Template Method (partiel) — Flux d'exécution des commandes . . . 10
4.6 Pattern Factory (partiel) — Création des formes . . . 10
4.7 Pattern Composite (partiel) — Hiérarchie des formes . . . 11
4.8 Interaction entre les patterns . . . 11
4.9 Diagramme de classes . . . 11
4.10 Chain of Responsibility — une réutilisation à trois niveaux . . . 12
4.10.1 Niveau géométrique — validation du redimensionnement . . . 12
4.10.2 Niveau stratégique — génération des formes par l'IA . . . 12
4.11 Pattern Strategy - IA . . . 13
4.12 Pattern Observer - Gestion Tour AI . . . 13
4.13 Pattern Builder et Chain of Responsibility - GameFlow . . . 14
4.14 Multithreading - AI . . . 14

5 Diagrammes de séquence **16**
5.1 Création d'une forme . . . 16
5.2 Undo / Redo . . . 17
5.3 Déplacement d'une forme . . . 17

5.4 Redimensionnement d'une forme . . . 17
5.5 Suppression d'une forme . . . 17
5.6 Pattern Observer (Modèle-Vue) . . . 18
5.7 Changement de stratégie . . . 18
5.8 Timer et disparition des formes rouges . . . 18
5.9 Mode 2 joueurs - Alternance des tours . . . 18
5.10 Niveau complet et transition . . . 18
5.11 CreateShapeCommand (détaillé) . . . 18

6 Implémentation **19**

7 Tests et validation **20**
7.1 Définitions des tests par module . . . 20
7.1.1 Module CommandHManager . . . 20
7.1.2 Module Command . . . 20
7.1.3 Module GameModel . . . 21
7.1.4 Module Shape . . . 22
7.1.5 Module Strategy . . . 22
7.1.6 Module TurnCoordinator . . . 22
7.1.7 Module View . . . 23
7.1.8 Récapitulatif des tests . . . 23
7.1.9 Test fin de partie correct . . . 23
7.2 Validation des patterns . . . 23

8 Analyse de qualité du code **25**
8.1 Présentation de l'outil . . . 25
8.2 Résultats obtenus . . . 25
8.3 Interprétation des résultats . . . 25
8.3.1 Points positifs . . . 25
8.3.2 Points d'amélioration . . . 26
8.4 Conclusion sur la qualité du code . . . 26

Conclusion **27**

Bibliographie **28**

Introduction

Ce projet porte sur le développement d'une application interactive fondée sur l'architecture MVC et plusieurs design patterns. Le système prend la forme d'un jeu dans lequel un joueur manipule des formes géométriques bleues dans une zone déjà occupée par des formes rouges, avec des contraintes de non-intersection et un score fondé sur la surface occupée.

L'objectif n'était pas seulement de produire une application fonctionnelle, mais aussi de mettre en œuvre une conception logicielle claire, modulaire et évolutive. Les patterns **Observer**, **State**, **Command**, **Strategy**, **Chain of Responsability**, **Builder** et **Abstract Factory** ont été intégrés entre autre afin de répondre à des besoins précis d'architecture et d'interaction.

Chapitre 1

Analyse du besoin

Le jeu consiste à placer et manipuler des formes dans une zone contrainte. Les formes rouges sont générées automatiquement et les formes bleues sont créées par le joueur.

1.1 Fonctionnalités principales

Les principales fonctionnalités sont :

- création de formes (rectangles et cercles) ;
- déplacement, redimensionnement et suppression ;
- contrôle des intersections avec les formes rouges ;
- calcul du score basé sur la surface des formes ;
- progression par niveaux (5 niveaux de difficulté).

1.2 Fonctionnalités détaillées

1.2.1 Gestion des formes

- **Création** : Les formes bleues sont créées par glisser-déposer (*drag and drop*) avec affichage d'un aperçu en temps réel.
- **Déplacement** : Une forme bleue se déplace en cliquant sur sa zone centrale et en glissant la souris.
- **Redimensionnement** : Une forme bleue se redimensionne en cliquant sur son bord et en glissant la souris.
- **Suppression** : Une forme sélectionnée peut être supprimée via un bouton dédié.
- **Sélection** : La forme cliquée est mise en évidence par un contour noir.

1.2.2 Contraintes et validation

- Les formes bleues ne doivent pas chevaucher les formes rouges.
- En mode solo, les formes bleues ne doivent pas se chevaucher entre elles.
- En mode 2 joueurs, le chevauchement entre formes bleues est autorisé.
- Toute forme doit rester entièrement dans l'aire de jeu.
- Taille minimale : 10 pixels pour les rectangles, rayon supérieur à 5 pixels pour les cercles.
- 4 formes bleues maximum peuvent être placées par niveau.
- En cas de collision lors d'un déplacement ou redimensionnement, la forme retourne à sa position ou à sa taille d'origine.

1.2.3 Niveaux de jeu

Niveau	Formes rouges	Temps	Difficulté
1	2	10s	Facile
2	3	10s	Moyen
3	4	8s	Difficile
4	5	8s	Très difficile
5	6	6s	Extrême

Table 1.1 : Configuration des niveaux

1.2.4 Stratégies de génération des formes rouges

- **RandomGenerationStrategy** : génération aléatoire des positions et des tailles ;
- **AIPlayerStrategy** : stratégie adaptée au mode contre l'ordinateur. L'IA génère ses propres formes en fonction de l'état du jeu, offrant un défi progressif au joueur.
- **TwoPlayerStrategy** : stratégie adaptée au mode deux joueurs.

1.2.5 Annulation et rétablissement (*Undo/Redo*)

- **Undo** : annule la dernière action (création, suppression, déplacement, redimensionnement).
- **Redo** : rétablit une action précédemment annulée.
- Les commandes sont stockées dans deux piles : `undoStack` et `redoStack`.

1.2.6 Calcul du score

- **Rectangle** : largeur $\times$ hauteur
- **Cercle** : $\pi \times \text{rayon}^2$
- **Score total** : somme des surfaces de toutes les formes bleues placées.

1.2.7 Timer et disparition des formes rouges

- Les formes rouges sont visibles pendant un temps limité, défini par le niveau.
- À l'expiration du timer, les formes rouges disparaissent automatiquement.
- Le joueur peut alors placer ses formes bleues.
- En mode solo, si moins de 4 formes sont placées avant la disparition, la partie est perdue.

Chapitre 2

Étude fonctionnelle

L'acteur principal est le joueur, qui interagit avec l'application par la souris.
Les principales actions possibles sont :

— démarrer une partie ;

— créer une forme ;

— déplacer, redimensionner ou supprimer une forme ;

— annuler et rétablir une action ;

— consulter le score.

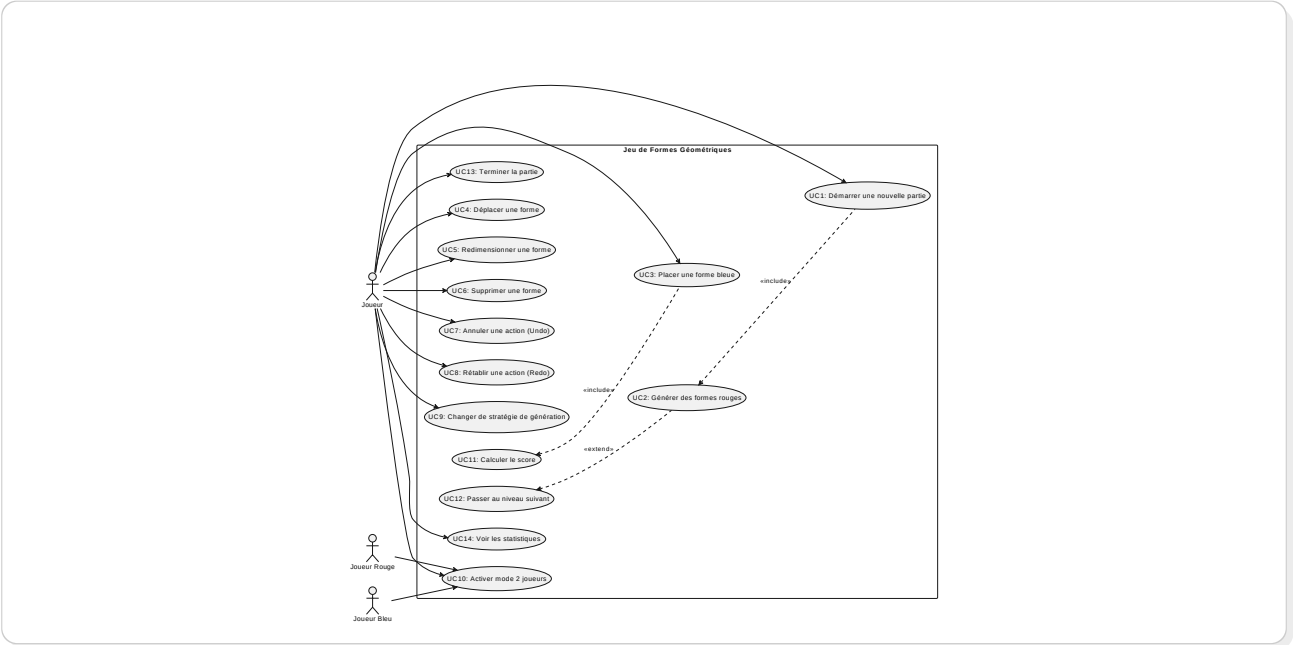


Figure 2.1 : Diagramme de cas d'utilisation

Chapitre 3

Conception générale

Le système est organisé selon l'architecture MVC :

- **Model** : gestion des formes, du score et des règles du jeu ;
- **View** : affichage graphique et interface utilisateur ;
- **Controller** : gestion des événements et des interactions.

Les design patterns suivants ont été utilisés :

- **Observer** pour synchroniser le modèle et la vue ;
- **State** pour gérer les modes d'interaction ;
- **Command** pour encapsuler les actions et gérer *undo/redo* ;
- **Strategy** pour la génération des formes rouges ;
- **Template Method** (partiel) pour structurer le flux d'exécution des commandes ;
- **Factory** (partiel) pour la création des formes ;
- **Composite** (partiel) pour manipuler uniformément les formes via une interface commune.
- **Builder** pour construire les chaînes de phases (GameFlowBuilder) ;
- **Chain of Responsibility** pour valider le redimensionnement et pour le pipeline de génération de l'IA ;
- **Abstract Factory** pour gérer les thèmes visuels (Dark et Light).

Ces choix permettent d'obtenir un système modulaire, lisible et maintenable.

Chapitre 4

Conception détaillée

4.1 Pattern Observer — Pour synchronisation Modèle-Vue

Le pattern **Observer** établit une relation de dépendance un-à-plusieurs entre le modèle et les vues. Lorsque l'état du modèle change, tous les observateurs enregistrés sont automatiquement notifiés et mis à jour.

Fonctionnement :

1. La GameModel étend la classe Observable de Java. Elle contient une liste d'observateurs.
2. La GameView implémente l'interface Observer et s'enregistre auprès du modèle via addObserver().
3. Lorsqu'une action modifie l'état du modèle (ajout, suppression, déplacement, etc.), le modèle appelle modelChanged(String event).
4. Cette méthode exécute setChanged() puis notifyObservers(event).
5. Tous les observateurs reçoivent l'événement via leur méthode update(Observable, Object).
6. La vue réagit en fonction de l'événement : repaint() pour mettre à jour l'affichage, affichage d'une popup pour GAME$_OVER, etc.$

Avantages :

- Découplage total entre le modèle et la vue.
- Plusieurs vues peuvent observer le même modèle.
- Facilite l'ajout de nouvelles interfaces (console, web, etc.).

4.2 Pattern State — Gestion des interactions souris

Le pattern **State** permet à un objet de modifier son comportement lorsque son état interne change. Ici, il gère les différents modes d'interaction souris.

Fonctionnement :

1. L'interface EtatInteraction déclare les méthodes liées aux événements souris : sourisAppuyee(), sourisDeplacee(), sourisRelachee(), sourisBougee().
2. Les classes concrètes implémentent ces méthodes selon leur comportement spécifique :
   - `EtatSelection` : attend la sélection d'une forme.
   - `EtatCreationRectangle` : permet de dessiner un rectangle par glisser-déposer.
   - `EtatCreationCercle` : permet de dessiner un cercle par glisser-déposer.
   - `EtatMoveShape` : gère le déplacement d'une forme.
   - `EtatResizeShape` : gère le redimensionnement.
3. Le ControleurSouris (contexte) maintient une référence vers l'état courant.

4. Lorsqu'un événement souris se produit, le contrôleur délègue l'appel à l'état courant.
5. Pour changer de comportement, il suffit d'appeler changerEtat(nouvelEtat).

Avantages :

— Élimine les longues structures conditionnelles (switch/case sur les modes).

— Chaque état est encapsulé dans sa propre classe.

— Facilite l'ajout de nouveaux modes d'interaction.

4.3 Pattern Command — Undo/Redo

Le pattern **Command** encapsule une requête sous forme d'objet, permettant de paramétrer, stocker, annuler et rétablir des actions.

Fonctionnement :

1. L'interface Command déclare trois méthodes : `execut()`, `undo()`, `redo()`.
2. Les commandes concrètes (CreateShapeCommand, DeleteShapeCommand, MoveShapeCommand, ResizeShapeCommand) implémentent ces méthodes.
3. Le CommandHManager gère deux piles :
   — `undoStack` : contient les commandes exécutées.
   — `redoStack` : contient les commandes annulées.
4. Lorsqu'une action est effectuée :
   (a) Une commande est instanciée avec le modèle et les paramètres nécessaires.
   (b) (command) appelle execut() puis empile la commande dans `undoStack`.
   (c) La pile redoStack est vidée.
5. Pour annuler :
   (a) La dernière commande est dépilée de undoStack.
   (b) Sa méthode undo() est appelée.
   (c) La commande est empilée dans redoStack.
6. Pour rétablir :
   (a) La dernière commande est dépilée de redoStack.
   (b) Sa méthode redo() est appelée.
   (c) La commande est empilée dans undoStack.

Avantages :

— Séparation entre l'émetteur et le récepteur de la requête.

— Support natif de l'undo/redo.

— Possibilité de stocker un historique illimité.

4.4 Pattern Strategy — Génération des formes rouges

Le pattern **Strategy** définit une famille d'algorithmes interchangeables. Il permet de changer dynamiquement la façon dont les formes rouges sont générées.

Fonctionnement :

1. L'interface ShapeGenerationStrategy déclare la méthode generateShapes(int count, int width, int height).

2. Trois stratégies concrètes sont implémentées :
   — RandomGenerationStrategy : génère des formes à positions et tailles aléatoires.
   — AIPlayerStrategy : adaptée au mode contre l'ordinateur (IA).
   — TwoPlayerStrategy : adaptée au mode 2 joueurs.
3. La GameModel contient une référence vers la stratégie courante.
4. L'utilisateur peut changer de stratégie via l'interface.
5. Lors de l'appel à generateRedShapes(), le modèle délègue la création à la stratégie.

Avantages :

— Les algorithmes sont interchangeables à l'exécution.
— Évite les conditions multiples pour choisir le comportement.
— Facilite l'ajout de nouvelles stratégies (ex : génération par fichier de configuration).

4.5 Pattern Template Method (partiel) — Flux d'exécution des commandes

Le pattern **Template Method** définit le squelette d'un algorithme dans une méthode, en déléguant certaines étapes à des sous-classes.

Fonctionnement :

1. La classe CommandHManager définit une méthode executeAndStore(Command) qui structure le flux :
   (a) Appel à command.execut().
   (b) Empilement dans undoStack.
   (c) Vidage de redoStack.
2. Les étapes spécifiques (ce que fait execut()) sont définies dans les commandes concrètes.
3. Les méthodes undo() et redo() suivent également un template similaire.

Avantages :

— Réutilise la structure commune de gestion des commandes.
— Évite la duplication de code dans le gestionnaire.
— Garantit un comportement cohérent pour toutes les commandes.

4.6 Pattern Factory (partiel) — Création des formes

Le pattern **Factory** délègue la création d'objets à des méthodes spécialisées.

Fonctionnement :

1. La ClickPlacementStrategy contient des méthodes factory :
   — createRectangleFromClicks(startX, startY, endX, endY, color)
   — createCircleFromClicks(centerX, centerY, edgeX, edgeY, color)
2. Ces méthodes calculent les dimensions à partir des points de clic et instancient la forme appropriée.
3. Les états de création (EtatCreationRectangle, EtatCreationCercle) utilisent ces factories.

Avantages :

— Centralise la logique de création des formes.
— Permet d'ajouter facilement de nouveaux types de formes.

— Sépare la construction de l'utilisation.

4.7 Pattern Composite (partiel) — Hiérarchie des formes

Le pattern **Composite** permet de traiter des objets individuels et des compositions d'objets de manière uniforme.

Fonctionnement :

1. L'interface GameShape définit les opérations communes à toutes les formes :
   — intersects(GameShape) : détection de collision.
   — getArea() : calcul de la surface.
   — contains(Point2D) : test d'appartenance.
   — move(), resize(), `clone()`, etc.
2. Les classes Rectangle et Circle implémentent cette interface.
3. Le modèle manipule les formes via l'interface GameShape, sans distinction de type.

Avantages :

— Uniformité du traitement des formes.
— Ajout facile de nouveaux types de formes (Triangle, Polygone, etc.).
— Les collections (List<GameShape>) peuvent contenir n'importe quelle forme.

4.8 Interaction entre les patterns

La figure **4.1** montre comment les différents patterns travaillent ensemble :

— Le ControleurSouris utilise le pattern **State** pour gérer les modes d'interaction.
— Les états créent des **Command** qui sont stockées par le **CommandHManager**.
— Les commandes modifient la **GameModel**.
— La **GameModel** utilise le pattern **Observer** pour notifier la **GameView**.
— La **GameModel** utilise le pattern **Strategy** pour générer les formes rouges.
— Les formes manipulées implémentent le pattern **Composite**.
— La création des formes utilise des méthodes **Factory**.

4.9 Diagramme de classes

Le diagramme de classes complet est présenté à la page suivante. Il illustre l'ensemble des relations entre les différentes classes et interfaces du système, ainsi que l'application des sept patterns de conception décrits ci-dessus.

graphicx

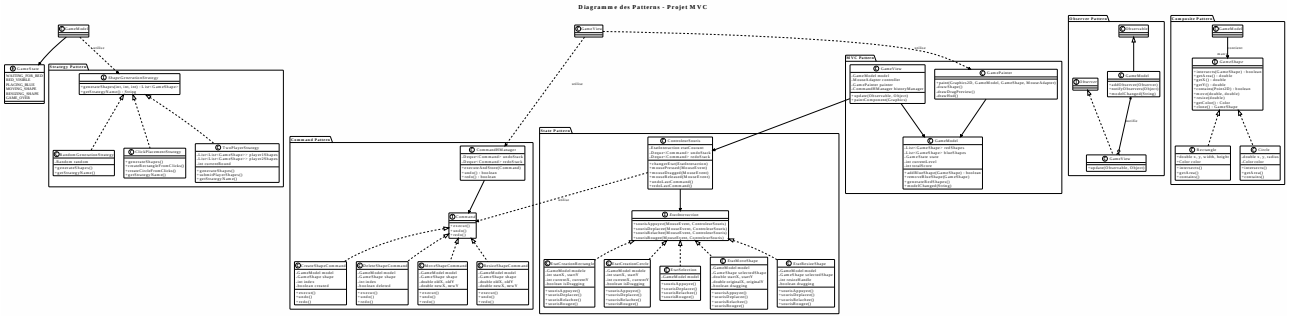


Figure 4.1 : Diagram Class

La classe centrale est GameModel, qui stocke l'état du jeu, les formes et le score. Les formes sont modélisées à partir d'une abstraction commune GameShape, spécialisée en Circle et Rectangle.

Le contrôleur principal, ControleurSouris, s'appuie sur le pattern **State** afin de déléguer les comportements à des objets spécialisés selon le mode courant. Les actions utilisateur sont encapsulées par le pattern **Command**.

4.10 Chain of Responsibility — une réutilisation à trois niveaux

Le pattern **Chain of Responsibility** a été appliqué dans trois contextes différents du projet. Dans chaque cas, le principe est le même : une requête circule le long d'une chaîne de maillons, et chaque maillon décide soit de la traiter, soit de la transmettre au suivant. Mais l'objet de la requête, les maillons et l'objectif final changent.

4.10.1 Niveau géométrique — validation du redimensionnement

Ici, la requête est une tentative de redimensionnement d'une forme. La chaîne valide cette opération selon plusieurs règles indépendantes :

- **ValidateResizeFactorValidator** : refuse un facteur nul.
- **ValidateIntersectionValidator** : vérifie qu'après redimensionnement, la forme n'empiète pas sur une forme rouge.
- **ValidateGameAreaValidator** : vérifie que la forme reste dans l'aire de jeu.

Chaque validateur est indépendant, facile à tester et à réordonner. La chaîne s'arrête dès qu'une règle échoue. Ce découpage évite une longue série de `if` dans le code du redimensionnement.

4.10.2 Niveau stratégique — génération des formes par l'IA

L'IA utilise également une chaîne de responsabilité pour découper la génération des formes en étapes.

Fonctionnement :

1. AIShapeGenerationBuilder construit une chaîne de phases.
2. Les phases concrètes sont :
   - AIAnalyzeGameStatePhase : analyse l'état actuel du jeu.
   - AIDecideShapeSizePhase : décide la taille des formes à générer.
   - AIGenerateShapesPhase : génère les formes selon les décisions prises.
3. Chaque phase peut être réordonnée, ajoutée ou supprimée sans casser le reste.
4. La chaîne est appelée via shapeGenerator.generate(...).

Avantages :

- Chaque étape est isolée et testable indépendamment.

— On peut réordonner/ajouter une phase sans modifier la stratégie globale.
— Meilleure maintenabilité du code de l'IA.

Algo IA pour ajustement dynamique de la taille

L'IA utilise plusieurs algorithmes pour adapter dynamiquement la taille des formes bleues en fonction de l'espace disponible.

$$\text{surfaceDisponible} = \text{largeurPanneau} \times \text{hauteurPanneau}$$

$$\text{pressionEspace} = 1 - \frac{\text{surfaceDisponible}}{500000}$$

$$\text{tailleAjustee} = \text{tailleBase} \times (1 - \text{pressionEspace} \times 0,5)$$

Plus l'espace est restreint, plus la taille des formes est réduite. Lorsque l'IA échoue à placer une forme (intersection avec obstacles ou sortie de zone) :

— Après 50 échecs consécutifs, la taille cible est réduite de 15%
— La réduction continue jusqu'à une taille minimale de 5 pixels
— Cela augmente les chances de trouver un emplacement valide

Si la surface couverte dépasse 50%, l'IA adopte une stratégie défensive (formes plus petites).

4.11 Pattern Strategy - IA

Le pattern **Strategy** est combiné avec la chaîne de responsabilité pour l'IA.

Fonctionnement :

1. AIPlayerStrategy (Strategy) : orchestrateur haut niveau.
   — Décide la logique globale (mode humain/IA, score, rounds).
   — Appelle la chaîne de génération.
2. La chaîne de phases (Chain) : pipeline de génération bas niveau.
   — Analyser → décider taille → générer.
3. AIPlayerStrategy.generateShapes(...) appelle shapeGenerator.generate(...).

Avantages :

— Séparation claire des responsabilités (orchestration vs génération).
— Évite qu'une seule classe devienne monolithique.
— Chaque niveau peut évoluer indépendamment.

4.12 Pattern Observer - Gestion Tour AI

Le pattern **Observer** est utilisé avec un mécanisme de synchronisation pour la gestion des tours.

Fonctionnement :

1. PlayerTurnState définit les états possibles d'un tour :
   — HUMAN_GUI : joueur humain via l'interface graphique.
   — HUMAN_CLI : joueur humain via ligne de commande.

   - AI : joueur automatique.
   - WAITING : en attente.
   - COMPLETED : tour terminé.
2. TurnSignal représente les événements du tour.
3. TurnCoordinator coordonne la progression :
   - Démarre un tour avec startTurn().
   - Demande une action.
   - Marque une action terminée.
   - Permet aux threads d'attendre un état précis.

Avantages :

- L'interface Swing reste réactive pendant les tours.
- Le tour est synchronisé entre les différents threads.
- L'IA et le joueur ne se gênent pas mutuellement.

4.13 Pattern Builder et Chain of Responsibility - GameFlow

Le GameFlowBuilder combine deux patterns : le **Builder** pour la construction et la **Chain of Responsibility** pour l'exécution.

Problématique : Le jeu doit pouvoir s'exécuter selon différents déroulements :

- Mode classique : affichage des obstacles → joueur joue → calcul du score.
- Mode avec mémorisation : affichage → attente → masquage → joueur joue → score.
- Mode IA : affichage → phase IA → joueur joue → score.

Sans pattern, chaque mode nécessiterait une méthode différente avec du code dupliqué.

Avantages :

- Réutilisation du code des phases.
- Construction flexible et lisible des différents déroulements.
- Ajout facile de nouvelles phases sans modifier les existantes.
- Séparation entre la construction (Builder) et l'exécution (Chain of Responsibility).

4.14 Multithreading - AI

Le multithreading est utilisé dans le projet pour gérer la synchronisation des tours de jeu sans bloquer l'interface graphique. Cette approche permet de maintenir une expérience utilisateur fluide tout en coordonnant les actions entre le joueur humain et l'IA.

Problématique : Dans un jeu avec des tours alternés (mode 2 joueurs ou mode IA), plusieurs problèmes se posent :

- L'interface graphique Swing doit rester réactive pendant les tours.
- L'IA doit pouvoir "réfléchir" sans bloquer l'affichage.
- Les actions du joueur humain et de l'IA ne doivent pas interférer.
- Il faut pouvoir attendre qu'un joueur termine son action avant de passer au suivant.

Solution : Pour le résoudre on a utilise un système de synchronisation basé sur les moniteurs Java (synchronized, wait(), notifyAll()) à travers trois classes principales.

Avantages :

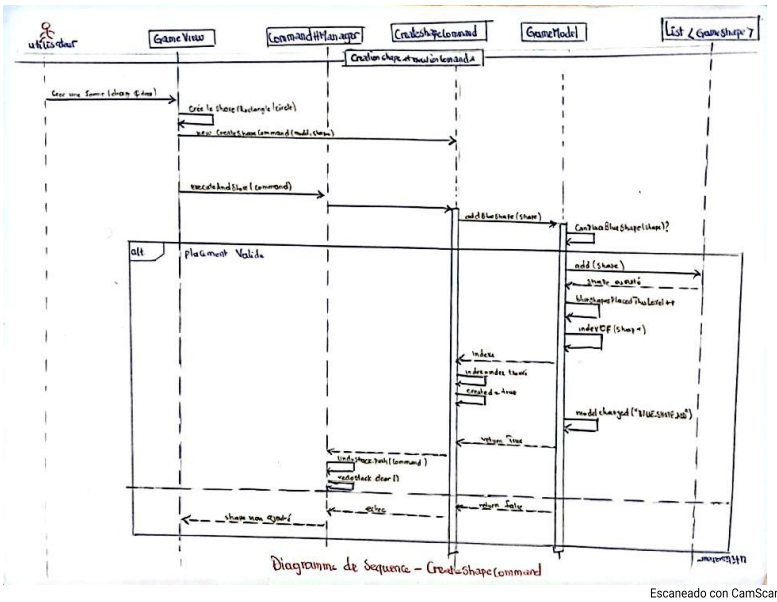
- Un thread dédié à l'attente des tours en mode 2 joueurs.
- Des Timer Swing pour l'IA et les animations.
- Une synchronisation adapter via TurnCoordinator.

Cette approche garantit une interface toujours réactive et une séparation claire entre la logique de jeu et la présentation.

Chapitre 5

Diagrammes de séquence

5.1 Création d'une forme



Escaneado con CamScanner

Figure 5.1 : Diagramme de séquence : création d'une forme

Lorsqu'un joueur choisit un mode de création et interagit avec la zone de jeu, l'événement est capté par la vue, transmis au contrôleur, puis délégué à l'état courant. Celui-ci construit une commande de création, qui modifie le modèle. Enfin, le modèle notifie la vue, ce qui provoque la mise à jour de l'affichage.

5.2 Undo / Redo

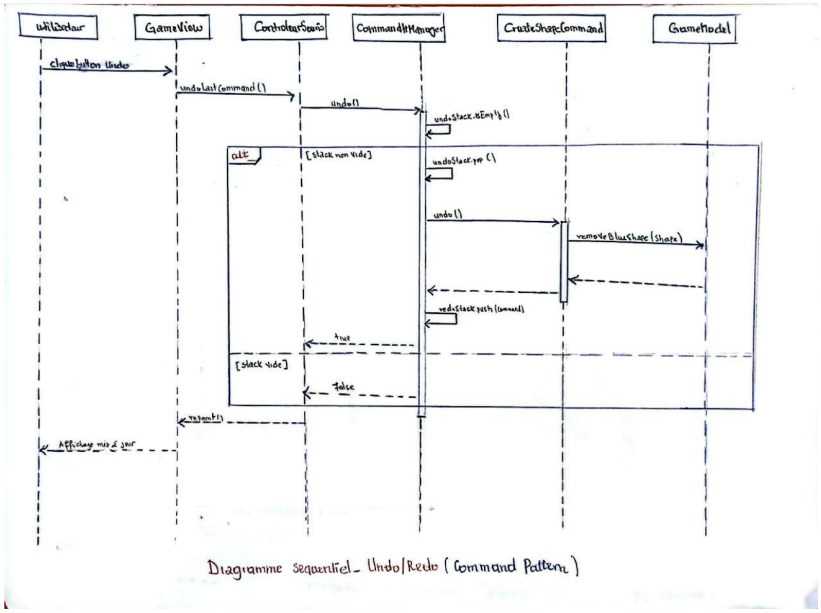


Figure 5.2 : Diagramme de séquence : undo / redo

Le mécanisme d'annulation et de rétablissement repose sur le pattern **Command**. Les actions sont stockées dans un historique, puis exécutées, annulées ou réappliquées selon la demande de l'utilisateur. Lors d'un Undo, la dernière commande est dépilée de l'undoStack et sa méthode undo() est appelée, puis la commande est déplacée vers le redoStack. Le Redo suit le principe inverse.

5.3 Déplacement d'une forme

Lorsque l'utilisateur clique sur la zone centrale (zone 1) d'une forme bleue, le contrôleur passe de l'état EtatSelection à l'état EtatMoveShape. Pendant le glissement, la forme est déplacée en temps réel. Au relâchement, le système vérifie les collisions avec les formes rouges, les autres formes bleues (mode solo) et les limites du jeu. En cas de collision, la forme retourne à sa position d'origine ; sinon, une MoveShapeCommand est créée et stockée dans l'historique.

5.4 Redimensionnement d'une forme

L'utilisateur clique sur le bord (zone 0) d'une forme sélectionnée, ce qui déclenche la transition vers l'état EtatResizeShape. Pendant le glissement, la forme est redimensionnée dynamiquement. Au relâchement, une validation est effectuée : si la nouvelle taille génère une collision ou sort de l'aire de jeu, la forme retrouve ses dimensions d'origine. Sinon, une ResizeShapeCommand est créée et ajoutée à l'historique.

5.5 Suppression d'une forme

L'utilisateur sélectionne une forme bleue puis clique sur le bouton "Eliminer". Une DeleteShapeCommand est créée et exécutée : la forme est supprimée du modèle, le compteur de formes placées est décrémenté et la vue est mise à jour. La commande est empilée dans l'undoStack. En cas d'Undo, la forme est restaurée à sa position d'origine via restoreBlueShape(). Le Redo la supprime à nouveau.

5.6 Pattern Observer (Modèle-Vue)

La GameView s'enregistre comme observateur auprès de la GameModel lors de son initialisation. Lorsqu'une action modifie l'état du modèle (ajout, suppression, déplacement, etc.), le modèle appelle modelChanged() qui déclenche setChanged() et notifyObservers(). Tous les observateurs enregistrés reçoivent l'événement via leur méthode update() et réagissent en rafraîchissant l'affichage.

5.7 Changement de stratégie

L'utilisateur peut changer dynamiquement la stratégie de génération des formes rouges via l'interface. La vue appelle `setGenerationStrategy()` du modèle qui met à jour sa référence. Lors de la prochaine génération (nouveau niveau), le modèle utilise cette stratégie pour créer les formes. Ce mécanisme illustre le pattern **Strategy**.

5.8 Timer et disparition des formes rouges

Au début de chaque niveau, le modèle programme un TimerTask avec un délai correspondant au temps imparti. À l'expiration, hideRedShapes() est appelée : les formes rouges disparaissent. En mode solo, si le joueur n'a pas placé les 4 formes bleues avant cette disparition, la partie est perdue. En mode 2 joueurs, le jeu passe simplement à la phase de placement.

5.9 Mode 2 joueurs - Alternance des tours

Après activation du mode 2 joueurs, le système alterne les tours entre le Joueur Rouge et le Joueur Bleu. Chaque joueur dispose de 4 tours maximum. À la fin du tour d'un joueur, completeTwoPlayerTurn() bascule le joueur actif, incrémente le compteur de tours et génère un nouveau set de formes rouges. Lorsque les 8 tours sont complétés, la partie se termine et les scores finaux sont affichés.

5.10 Niveau complet et transition

Lorsqu'un joueur place sa 4ème forme bleue, le modèle calcule le score. En mode solo, une popup de victoire propose de passer au niveau supérieur. Si l'utilisateur accepte, le modèle charge la configuration du niveau suivant, génère les nouvelles formes rouges et réinitialise le compteur. En mode 2 joueurs, le système alterne simplement le joueur actif.

5.11 CreateShapeCommand (détaillé)

Ce diagramme détaille le cycle de vie complet d'une CreateShapeCommand. La commande est instanciée avec le modèle et la forme à créer. Lors de l'exécution, la méthode addBlueShape() du modèle est appelée. Si le placement est valide, la forme est ajoutée à la liste, le compteur est incrémenté et la commande est empilée dans l'undoStack. L'Undo appelle removeBlueShape() et le Redo réapplique l'ajout.

Chapitre 6

Implémentation

L'application a été développée en Java avec Swing. Le code est réparti en plusieurs packages :

- `model.games`
- `model.game`
- `model.shapes`
- `model.commands`
- `model.strategy`
- `model.view`
- `model.controller`

Cette organisation reflète directement l'architecture définie lors de la phase de conception et permet de séparer les responsabilités de manière claire.

Chapitre 7

Tests et validation

Les tests ont porté sur :

- la création de formes ;
- la suppression ;
- le déplacement ;
- le redimensionnement ;
- le mécanisme *undo/redo* ;
- la gestion du modèle (niveaux, scores, timers, mode 2 joueurs) ;
- les opérations géométriques (intersections, aires, classification des zones) ;
- les stratégies de génération des formes rouges ;
- la synchronisation des tours (TurnCoordinator) ;
- les composants graphiques (thèmes visuels, menu, tutoriel, peintre).
- Test fin de match 2 Humain et IA.
- Test fin de match 2 Jouer.

7.1 Définitions des tests par module

7.1.1 Module CommandHManager

- **testExecuteAndStore** : Vérifie qu'une commande exécutée est correctement stockée dans l'historique et que la forme est ajoutée au modèle.
- **testUndo** : Vérifie que l'annulation d'une commande restaure l'état précédent et que la forme est retirée du modèle.
- **testRedo** : Vérifie que le rétablissement d'une commande annulée réapplique correctement l'action.
- **testClearRedoStack** : Vérifie que l'exécution d'une nouvelle commande vide la pile *redoStack*.
- **testUndoEmpty** : Vérifie que l'appel à *undo* retourne `false` lorsque l'historique est vide.
- **testRedoEmpty** : Vérifie que l'appel à *redo* retourne `false` lorsqu'aucune action n'est à rétablir.
- **testUndoRedoOrderWithMultipleCommands** : Vérifie l'ordre correct d'annulation et de rétablissement avec plusieurs commandes successives.
- **testNewCommandAfterUndoClearsRedoStack** : Vérifie qu'une nouvelle commande après un *undo* vide correctement la pile *redoStack*.

7.1.2 Module Command

- **testCreateShapeCommand** : Vérifie le cycle complet d'une commande de création (exécution → undo → redo).

- **testDeleteShapeCommand** : Vérifie le cycle complet d'une commande de suppression.
- **testMoveShapeCommand** : Vérifie le déplacement d'une forme et la sauvegarde des positions originales.
- **testResizeShapeCommandCircle** : Vérifie le redimensionnement d'un cercle avec conservation du facteur d'échelle.
- **testResizeShapeCommandRectangle** : Vérifie le redimensionnement d'un rectangle.
- **testResizeShapeCommandRectangleWithZeroFactor** : Vérifie qu'un facteur de redimensionnement nul ne modifie pas la forme.
- **testResizeChainRejectsIntersectionAfterResize** : Vérifie que la chaîne de validation refuse un redimensionnement causant une intersection avec une forme rouge.
- **testResizeChainRejectsOutOfGameAreaAfterResize** : Vérifie que la chaîne de validation refuse un redimensionnement sortant de l'aire de jeu.

7.1.3 Module GameModel

Tests de base

- **testAddBlueShape** : Vérifie l'ajout d'une forme bleue dans le modèle.
- **testAddBlueShapeWhenGameFinished** : Vérifie qu'aucune forme ne peut être ajoutée après la fin de la partie.
- **testAddBlueShapeIntersectsRed** : Vérifie qu'une forme bleue ne peut pas chevaucher une forme rouge.
- **testAddBlueShapeOutsideGameArea** : Vérifie qu'une forme ne peut pas être placée hors de l'aire de jeu.
- **testRemoveBlueShape** : Vérifie la suppression d'une forme bleue du modèle.
- **testRemoveBlueShapeDecrementsCounter** : Vérifie que le compteur de formes est décrémenté lors d'une suppression.
- **testFindBlueShapeAt** : Vérifie la recherche d'une forme par coordonnées.
- **testFindBlueShapeAtNotFound** : Vérifie qu'aucune forme n'est trouvée pour des coordonnées vides.
- **testCanPlaceBlueShapeIntersection** : Vérifie la détection de chevauchement entre formes bleues.
- **testCanPlaceBlueShapeNoIntersection** : Vérifie qu'un placement sans collision est autorisé.
- **testCalculateScore** : Vérifie le calcul du score basé sur la surface des formes.
- **testIsPointInsideGameArea** : Vérifie la détection d'un point dans l'aire de jeu.
- **testIsShapeWithinGameArea** : Vérifie qu'une forme est entièrement contenue dans l'aire de jeu.
- **testSetGameState** : Vérifie le changement d'état du jeu.
- **testEnableTwoPlayerMode** : Vérifie l'activation correcte du mode 2 joueurs.
- **testGetCurrentDrawingColor** : Vérifie la couleur associée au joueur actif (rouge ou bleu).
- **testGetCurrentDrawingColorInSoloMode** : Vérifie la couleur par défaut en mode solo (bleu).

Tests supplémentaires

- **testNextLevelIncrementsLevel** : Vérifie l'incrémentation du niveau.
- **testCantPlaceMoreThan4BlueShapesPerLevel** : Vérifie la limite de 4 formes par niveau.
- **testGetBlueShapesRemainingForLevel** : Vérifie le calcul des formes restantes.
- **testAreRedShapesVisibleInitially** : Vérifie la visibilité initiale des formes rouges.
- **testHideRedShapes** : Vérifie la disparition des formes rouges.
- **testShowRedShapes** : Vérifie la réapparition des formes rouges.
- **testGetRemainingRedTime** : Vérifie le temps restant du timer.

— **testRemoveNonExistentShape** : Vérifie la suppression d'une forme inexistante.
— **testRestoreBlueShapeWithInvalidIndex** : Vérifie la restauration avec un index invalide.
— **testPlaceShapeExactlyOnGameAreaBorder** : Vérifie le placement sur les bords.
— **testPlaceShapeExactlyOnBorderEdge** : Vérifie le placement aux limites exactes.
— **testGetLevelConfig** : Vérifie la configuration des niveaux.
— **testGetLevelConfigWithInvalidLevel** : Vérifie le comportement avec un niveau invalide.
— **testIsGameFinished** : Vérifie la détection de fin de partie.
— **testGetFinalCoveredArea** : Vérifie la surface totale couverte.
— **testGetStatistics** : Vérifie l'affichage des statistiques.
— **testSetGameAreaSize** : Vérifie le redimensionnement de l'aire de jeu.
— **testSetGameAreaSizeWithInvalidValues** : Vérifie le comportement avec des valeurs invalides.
— **testSetCurrentLevel** : Vérifie le changement de niveau manuel.

7.1.4 Module Shape

— **testCircleContains** : Vérifie la méthode `contains` pour Circle (centre, intérieur, extérieur).
— **testRectangleContains** : Vérifie la méthode `contains` pour Rectangle.
— **testCircleRectangleIntersection** : Vérifie l'intersection entre cercle et rectangle.
— **testCircleCircleIntersection** : Vérifie l'intersection entre deux cercles.
— **testRectangleRectangleIntersection** : Vérifie l'intersection entre deux rectangles.
— **testCircleZoneType** : Vérifie les zones (centre, bord, dehors) pour un cercle.
— **testRectangleZoneType** : Vérifie les zones pour un rectangle.
— **testCircleArea** : Vérifie le calcul de l'aire d'un cercle.
— **testRectangleArea** : Vérifie le calcul de l'aire d'un rectangle.
— **testCircleMove** : Vérifie le déplacement d'un cercle.
— **testRectangleResize** : Vérifie le redimensionnement d'un rectangle.
— **testCircleResize** : Vérifie le redimensionnement d'un cercle.

7.1.5 Module Strategy

— **generateShapesReturnsExpectedCount** : Vérifie que la stratégie aléatoire génère le bon nombre de formes rouges.
— **generateShapesWithNonPositiveCount** : Vérifie le comportement avec un nombre non positif.
— **strategyNameIsStable** : Vérifie que le nom de la stratégie est cohérent.

7.1.6 Module TurnCoordinator

— **startTurnSetsStateSignalAndTurnNumber** : Vérifie l'initialisation correcte d'un tour.
— **completeActionAndCompleteTurnTransitionCorrectly** : Vérifie les transitions d'état après action et fin de tour.
— **awaitStateUnblocksWhenExpectedStateIsReached** : Vérifie le déblocage d'un thread attendant un état spécifique.
— **awaitTurnCompletionUnblocksAfterCompleteAction** : Vérifie le déblocage d'un thread attendant la fin du tour.
— **abortTurnResetsStateAndSignal** : Vérifie la réinitialisation correcte en cas d'abandon.

7.1.7 Module View

- **gameCPanelButtonsAndCallbacks** : Vérifie que les boutons du panneau de contrôle déclenchent les bons callbacks.
- **themeFactoryAndButtonFactory** : Vérifie la création des thèmes et des boutons stylisés.
- **mainMenuDifficultyMapping** : Vérifie le mapping correct entre libellé de difficulté et niveau.
- **mainMenuStyleHelpers** : Vérifie les méthodes de style du menu principal.
- **menuSelectionRecordStoresValues** : Vérifie l'enregistrement des valeurs de sélection.
- **menuShowRejectsNullCallback** : Vérifie la validation du callback non nul.
- **gameTutorialPrivateHelpers** : Vérifie les méthodes privées du tutoriel.
- **gamePainterPaintNominal** : Vérifie le peintre en conditions normales.
- **gamePainterDragPreviewBranches** : Vérifie l'affichage des aperçus pendant le glissement.
- **gameViewControlButtonsFlow** : Vérifie le flux complet des boutons de contrôle.
- **gameViewPrivateHelpers** : Vérifie les méthodes privées de la vue.

7.1.8 Récapitulatif des tests

- **testCircleContains** : Vérifie l'appartenance d'un point à un cercle.
- **testRectangleContains** : Vérifie l'appartenance d'un point à un rectangle.
- **testCircleRectangleIntersection** : Vérifie la détection de collision entre un cercle et un rectangle.
- **testCircleCircleIntersection** : Vérifie la détection de collision entre deux cercles.
- **testRectangleRectangleIntersection** : Vérifie la détection de collision entre deux rectangles.
- **testCircleZoneType** : Vérifie la classification des zones (centre, bord, extérieur) pour un cercle.
- **testRectangleZoneType** : Vérifie la classification des zones pour un rectangle.
- **testCircleArea** : Vérifie le calcul de la surface d'un cercle.
- **testRectangleArea** : Vérifie le calcul de la surface d'un rectangle.
- **testCircleMove** : Vérifie le déplacement d'un cercle.
- **testRectangleResize** : Vérifie le redimensionnement d'un rectangle.
- **testCircleResize** : Vérifie le redimensionnement d'un cercle.
- **testShapeClone** : Vérifie que la copie d'une forme est indépendante de l'originale.

7.1.9 Test fin de partie correct

- **aiDoesNotGenerateAfterGameFinished** : Vérifie que l'IA ne génère plus de formes une fois la partie terminée. Le modèle est marqué comme fini, puis l'appel à `generateShapes()` retourne une liste vide.
- **twoPlayerAddBlueShapeAndBlockAfterEnd** : Vérifie l'ajout d'une forme bleue en mode 2 joueurs, puis s'assure qu'aucune forme ne peut être ajoutée après la fin de la partie.

7.2 Validation des patterns

Les tests permettent de valider le bon fonctionnement des patterns implémentés :

- **Pattern Command** : Les tests *Undo/Redo* valident l'encapsulation des actions et la gestion de l'historique. Les 8 tests de `CommandTest` et les 8 tests de `CommandHManagerTest` couvrent l'intégralité du cycle de vie des commandes.
- **Pattern Composite** : Les 13 tests de `ShapeTest` valident l'interface commune `GameShape` à travers les méthodes *intersects*, *contains*, *getArea*, *move*, *resize* et *clone*.

— **Pattern Observer** : Les tests fonctionnels (non unitaires) valident la synchronisation modèle-vue via les notifications `modelChanged()`.

— **Pattern Strategy** : Les tests de génération de formes rouges (dans `StrategyTest`) valident l'interchangeabilité des stratégies `RandomGenerationStrategy`, `ClickPlacementStrategy` et `TwoPlayerStrategy`.

— **Pattern State** : Les tests d'interaction (via le contrôleur) valident les transitions entre états : `EtatSelection`, `EtatCreationRectangle`, `EtatCreationCercle`, `EtatMoveShape`, `EtatResizeShape`.

— **Pattern MVC** : L'ensemble des tests valide l'architecture globale séparant le modèle (`GameModel`), la vue (`GameView`) et le contrôleur (`ControleurSouris`).

Chapitre 8

Analyse de qualité du code

8.1 Présentation de l'outil

Dans le cadre de l'évaluation de la qualité du code, une analyse a été réalisée à l'aide d'un outil d'inspection statique (type SonarQube). Cet outil permet d'évaluer différents aspects du code, notamment la sécurité, la fiabilité, la maintenabilité ainsi que la couverture des tests.

8.2 Résultats obtenus

Figure 8.1 : Résultat de l'analyse de qualité du code

Les résultats obtenus sont les suivants :

- **Sécurité :** niveau **A** — aucune vulnérabilité détectée ;
- **Fiabilité :** niveau **B** — **3** anomalies potentielles identifiées ;
- **Maintenabilité :** niveau **A** — **209** remarques d'amélioration du code (*code smells*) ;
- **Points critiques analysés : 100%** — tous les points sensibles ont été revus ;
- **Couverture des tests : 0,0%** — non mesurée par l'outil ;
- **Duplications : 1,8%** — taux très faible, signe d'un code bien structuré.

8.3 Interprétation des résultats

L'analyse met en évidence un **bon niveau global de qualité du code**.

8.3.1 Points positifs

- **Sécurité (niveau A)** : Aucune vulnérabilité n'a été détectée, ce qui constitue un point particulièrement rassurant pour l'intégrité du jeu.

— **Maintenabilité (niveau A)** : Le code est globalement bien structuré et facile à maintenir.

— **Duplications (1,8%)** : Ce taux est **très faible** (largement en dessous du seuil acceptable de 5%), ce qui indique une bonne factorisation du code et l'absence de redondances inutiles.

— **Hotspots analysés (100%)** : Tous les points critiques du code ont été revus, garantissant une sécurité maximale.

8.3.2 Points d'amélioration

— **Fiabilité (niveau B)** : La présence de **5 anomalies** suggère des points de vigilance dans certaines parties du code. Ces anomalies méritent d'être corrigées pour renforcer la robustesse du système.

— **Code smells (209)** : Bien que la maintenabilité soit notée A, le nombre élevé de *code smells* indique des possibilités d'optimisation pour améliorer la lisibilité et la structure du programme.

— **Couverture des tests (0,0%)** : Cette valeur **ne reflète pas l'absence de tests**. Comme expliqué dans le chapitre précédent, des tests unitaires ont été écrits avec JUnit (71 tests au total). Cependant, SonarQube n'a pas été configuré pour intégrer les résultats de ces tests.

8.4 Conclusion sur la qualité du code

L'analyse SonarQube démontre que le projet atteint un **bon niveau de qualité globale** :

— Sécurité et maintenabilité au plus haut niveau (A) ;

— Taux de duplication exceptionnellement bas (1,8%) ;

— Tous les points critiques ont été analysés (100%).

Les axes d'amélioration identifiés (3 anomalies, 209 code smells, couverture à configurer) ne remettent pas en cause la qualité du livrable, mais constituent des pistes pour une version future. Cette analyse constitue une étape importante dans la validation de la qualité du projet et met en évidence les points forts ainsi que les axes de progression.

Conclusion

Dans ce projet, nous avons réalisé une application interactive basée sur l'architecture MVC. On a utilisé plusieurs design patterns comme Command, State, Observer et Strategy.

Ces patterns nous ont aidés à mieux organiser le code et à séparer les différentes responsabilités. Par exemple, le pattern Command permet de gérer facilement les actions undo et redo, et le pattern State facilite la gestion des interactions avec la souris.

L'application obtenue fonctionne correctement et reste assez simple à comprendre et à modifier. Nous avons aussi utilisé des diagrammes pour expliquer le fonctionnement du système.

Ce projet nous a permis de mieux comprendre comment utiliser les design patterns dans une application réelle. Il nous a aussi aidés à améliorer nos compétences en programmation orientée objet.

En général, cette expérience a été utile pour apprendre à concevoir une application plus structurée.

Bibliographie

E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns : Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

Oracle, *Java Platform Documentation*, https://docs.oracle.com/javase/8/docs/

Krasner, G., Pope, S., *A Description of the Model-View-Controller User Interface Paradigm*, Journal of Object-Oriented Programming, 1988.

PlantUML, *Official Documentation*, https://plantuml.com

SonarSource, *SonarQube Documentation*, https://www.sonarsource.com/products/sonarqube/